

Manual de Instalación y Configuración del Sistema de Auditoría Inteligente (SAI-CGN)

1. Introducción

Este manual proporciona las instrucciones detalladas para la instalación, configuración y despliegue del Sistema de Auditoría Inteligente (SAI-CGN) en la infraestructura de producción de la Contraloría General de la Nación (CGN). El objetivo es guiar al personal técnico de la CGN a través de los pasos necesarios para poner en marcha el sistema, asegurando su correcto funcionamiento y cumplimiento de los requisitos técnicos y de seguridad establecidos en la Especificación de Requisitos de Software (ERS) [1].

2. Requisitos del Sistema

2.1. Requisitos de Hardware

El sistema SAI-CGN operará en la infraestructura tecnológica de la CGN (nube privada / híbrida) [1]. Se requiere la siguiente configuración mínima para los nodos del clúster de Kubernetes:

Componente	Requisito Mínimo
CPU	8 vCPUs
RAM	32 GB
Almacenamiento	500 GB SSD de alto rendimiento
Red	Conectividad de 10 Gbps

2.2. Requisitos de Software

- **Sistema Operativo:** Linux (Ubuntu Server 20.04 LTS o superior recomendado).
- **Contenedores:** Docker Engine (versión 20.10 o superior).
- **Orquestación:** Kubernetes (versión 1.24 o superior).
- **Base de Datos:** PostgreSQL (versión 14 o superior) o compatible con el SMBD de la CGN.
- **Herramientas de Red:** `kubectl`, `helm`, `docker-compose` (para entornos de desarrollo/pruebas locales).
- **Seguridad:** Certificados TLS 1.3 para cifrado en tránsito.

3. Preparación del Entorno

3.1. Aprovisionamiento de Infraestructura

El área de TI de la CGN debe aprovisionar la infraestructura de servidores en la nube privada/híbrida, asegurando que cumpla con los requisitos de hardware y red. Se recomienda un clúster de Kubernetes con al menos 3 nodos de trabajo para alta disponibilidad y escalabilidad.

3.2. Instalación y Configuración de Docker y Kubernetes

1. Instalación de Docker Engine en cada nodo del clúster:

```

sudo apt update
sudo apt install apt-transport-https ca-certificates curl software-properties-common
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io
sudo usermod -aG docker $USER
newgrp docker

```

2. Instalación de Kubernetes (kubeadm, kubelet, kubectl) en cada nodo:

```

sudo apt update
sudo apt install -y apt-transport-https curl
curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo gpg --dearmor -o /usr/share/keyrings/kubernetes-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/kubernetes-archive-keyring.gpg] https://apt.kubernetes.io/ kubernetes-xenial main" | sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt update
sudo apt install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl

```

3. Inicialización del clúster de Kubernetes (solo en el nodo maestro):

```

sudo kubeadm init --pod-network-cidr=10.244.0.0/16
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml

```

4. Unir nodos de trabajo al clúster (en cada nodo de trabajo): Ejecutar el comando `kubeadm join` proporcionado por la salida de `kubeadm init` en el nodo maestro.

3.3. Configuración de Bases de Datos

El SAI-CGN requiere una base de datos PostgreSQL. Se recomienda desplegarla como un StatefulSet en Kubernetes para alta disponibilidad y persistencia de datos.

1. Creación de PersistentVolume y PersistentVolumeClaim para PostgreSQL:

```
# postgres-pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: postgres-pv
  labels:
    type: local
spec:
  storageClassName: manual
  capacity:
    storage: 100Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data/postgres"
---
# postgres-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 100Gi
```

Aplicar con `kubectl apply -f postgres-pv.yaml` y `kubectl apply -f postgres-pvc.yaml`.

2. Despliegue de PostgreSQL en Kubernetes:

```

# postgres-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
  labels:
    app: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:14
          env:
            - name: POSTGRES_DB
              value: "saicgn_db"
            - name: POSTGRES_USER
              value: "saicgn_user"
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: password
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-pvc

```

Crear un secreto para la contraseña de la base de datos: `kubectl create secret generic postgres-secret --from-`

```
literal=password='your_strong_password' Aplicar el despliegue con kubectl  
apply -f postgres-deployment.yaml.
```

3. Exposición del servicio PostgreSQL:

```
# postgres-service.yaml  
apiVersion: v1  
kind: Service  
metadata:  
  name: postgres-service  
spec:  
  selector:  
    app: postgres  
  ports:  
    - protocol: TCP  
      port: 5432  
      targetPort: 5432
```

Aplicar con `kubectl apply -f postgres-service.yaml`.

3.4. Integraciones con SECOP II, CHIP y SIRECI

El SAI-CGN se conectará a SECOP II, CHIP y SIRECI mediante API REST [1]. Se deben configurar las credenciales y los endpoints de estas APIs en el sistema.

1. **Obtención de Credenciales:** Contactar a los administradores de SECOP II, CHIP y SIRECI para obtener las claves API, tokens de acceso y URLs de los servicios.
2. **Configuración de Secretos en Kubernetes:** Almacenar las credenciales de forma segura como secretos de Kubernetes.

```
kubectl create secret generic secop-chip-sireci-credentials \  
  --from-literal=SECOP_API_KEY='your_secop_api_key' \  
  --from-literal=CHIP_API_KEY='your_chip_api_key' \  
  --from-literal=SIRECI_API_KEY='your_sireci_api_key'
```

3. **Configuración de Variables de Entorno:** Inyectar estos secretos como variables de entorno en los pods del SAI-CGN que requieran acceso a estas integraciones.

4. Despliegue del Sistema SAI-CGN en Kubernetes

El SAI-CGN se desplegará como una colección de microservicios en Kubernetes. Se utilizarán manifiestos de despliegue (Deployment, Service, Ingress) para cada módulo.

4.1. Estructura de Directorios del Proyecto

Se asume una estructura de directorios similar a la siguiente:

```
/home/ubuntu/saicgn/  
├── manifests/  
│   ├── module01-deployment.yaml  
│   ├── module01-service.yaml  
│   ├── ...  
│   └── ingress.yaml  
├── dockerfiles/  
│   ├── module01/Dockerfile  
│   ├── module02/Dockerfile  
│   └── ...  
└── src/  
    ├── module01/  
    ├── module02/  
    └── ...
```

4.2. Construcción de Imágenes Docker

Para cada módulo del SAI-CGN, se debe construir una imagen Docker.

```
cd /home/ubuntu/saicgn/dockerfiles/module01  
docker build -t saicgn/module01:latest .  
# Repetir para cada módulo (module02, module03, etc.)
```

Se recomienda etiquetar las imágenes con versiones específicas y subirlas a un registro de contenedores privado (ej. Docker Hub privado, Google Container Registry, Azure Container Registry).

4.3. Despliegue de Módulos en Kubernetes

1. Despliegue del Módulo M08 — Seguridad y Auditoría de Sistema:

```
# manifests/module08-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: saicgn-module08
  labels:
    app: saicgn-module08
spec:
  replicas: 2
  selector:
    matchLabels:
      app: saicgn-module08
  template:
    metadata:
      labels:
        app: saicgn-module08
    spec:
      containers:
        - name: module08
          image: saicgn/module08:latest # Reemplazar con la imagen
correcta
          ports:
            - containerPort: 8080
          env:
            - name: DATABASE_URL
              value:
                "postgresql://saicgn_user:${POSTGRES_PASSWORD}@postgres-
                service:5432/saicgn_db"
            # ... otras variables de entorno para configuración de
            seguridad
```

Aplicar con `kubectl apply -f manifests/module08-deployment.yaml`.

2. Despliegue del Módulo M01 — Gestión de Usuarios y Roles:


```
# manifests/module01-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: saicgn-module01
  labels:
    app: saicgn-module01
spec:
  replicas: 2
  selector:
    matchLabels:
      app: saicgn-module01
  template:
    metadata:
      labels:
        app: saicgn-module01
    spec:
      containers:
        - name: module01
          image: saicgn/module01:latest
          ports:
            - containerPort: 8080
          env:
            - name: DATABASE_URL
              value: "postgresql://saicgn_user:${POSTGRES_PASSWORD}@postgres-
service:5432/saicgn_db"
            - name: SSO_PROVIDER_URL
              value: "https://sso.cgn.gov.co/auth"
            # ... otras variables de entorno para configuración de SSO
```

Aplicar con `kubectl apply -f manifests/module01-deployment.yaml`.

- Despliegue de los demás módulos (M02 a M07) siguiendo un patrón similar.** Asegurarse de configurar las variables de entorno para las integraciones (SECOP II, CHIP, SIRECI) en los módulos correspondientes.

4.4. Configuración de Ingress

Se utilizará un Ingress Controller (ej. Nginx Ingress Controller) para exponer los servicios del SAI-CGN al exterior y gestionar el tráfico HTTP/HTTPS.

1. Instalación del Nginx Ingress Controller (si no está instalado):

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install ingress-nginx ingress-nginx/ingress-nginx
```

2. Creación de un Ingress para el SAI-CGN:

```
# manifests/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: saicgn-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - saicgn.cgn.gov.co
      secretName: saicgn-tls-secret # Secreto con el certificado TLS
  rules:
    - host: saicgn.cgn.gov.co
      http:
        paths:
          - path: /module01
            pathType: Prefix
            backend:
              service:
                name: saicgn-module01-service
                port:
                  number: 8080
          - path: /module02
            pathType: Prefix
            backend:
              service:
                name: saicgn-module02-service
                port:
                  number: 8080
    # ... agregar paths para los demás módulos
```

```
Crear un secreto TLS con el certificado para saicgn.cgn.gov.co: kubectl  
create secret tls saicgn-tls-secret --key /path/to/tls.key --cert  
/path/to/tls.crt Aplicar el Ingress con kubectl apply -f  
manifests/ingress.yaml.
```

5. Configuración Post-Instalación

5.1. Configuración Inicial del Sistema

- **Creación de Usuario Administrador:** Acceder a la interfaz del Módulo M01 para crear el primer usuario administrador del sistema.
- **Configuración de Roles y Permisos:** Definir los roles y permisos específicos para cada tipo de usuario (Auditor de Campo, Auditor Líder, Analista de Datos, etc.) según el RBAC.
- **Configuración de Fuentes de Datos:** En el Módulo M02, configurar las fuentes de datos fiscales y programar las tareas de ingesta.
- **Configuración de Modelos de IA:** En el Módulo M03, verificar la carga de los modelos de IA y configurar los parámetros iniciales.
- **Indexación de Normativa:** En el Módulo M04, asegurar que el corpus normativo esté correctamente indexado para el asistente conversacional.

5.2. Monitoreo y Logging

Se recomienda integrar el clúster de Kubernetes con herramientas de monitoreo y logging (ej. Prometheus, Grafana, ELK Stack) para supervisar el rendimiento del sistema y detectar posibles problemas.

6. Plan de Rollback

En caso de problemas críticos durante el despliegue o la configuración, se debe seguir el Plan de Rollback detallado en el Plan de Implementación y Despliegue [2].

7. Referencias

[1] Especificación de Requisitos de Software (ERS) del Sistema de Auditoría Inteligente (SAI-CGN), versión 1.0. [2] Plan de Implementación y Despliegue del Sistema de Auditoría Inteligente (SAI-CGN), versión 1.0.